



Guion de Cristofer — Checkout distribuido + Demo

Tu parte de la presentación de Krypton. Hablás en **5 slides**, en **dos momentos**: el corazón del proyecto, el checkout (slides 14–16), y después la demo en vivo + conclusiones (slides 22–23).

Cómo usar esto: el texto entre «**comillas**» es para **LEER** en voz alta. Mejor todavía: aprendételo y decilo natural. Lo de "En pantalla" es lo que se ve en la diapo (pocas palabras); el detalle lo ponés vos al hablar.

Consejo: la **demo (slide 22) es tuya y es lo que más impacta**. Ensayala aparte varias veces y tené un video grabado como plan B por si falla la red.

Tus turnos: Slides **14, 15, 16** (checkout) y **22, 23** (demo + conclusiones). **Duración:** ~5 min + demo.

Slide 14 — El problema del checkout

En pantalla: El checkout debe **descontar stock**... pero el stock vive en **catalog** (otra base, otro proceso) · No hay `@Transactional` que cruce dos servicios · ¿"todo o nada"?

Decís:

«Gracias, Curi. Yo les voy a contar el corazón del proyecto: el checkout. Y arranco con el problema. Cuando alguien compra, el checkout tiene que descontar el stock. Pero el stock no vive acá: vive en el catálogo, que es otro servicio, otro proceso y otra base de datos. Y acá está el lío: no existe un `@Transactional` que cruce dos servicios. En un monolito esto era una sola transacción, todo o nada. Acá no. Entonces, ¿cómo garantizo que sea todo o nada?»

Slide 15 — Feign + Saga con compensación

En pantalla: **Feign** = llamar a otro servicio como un método (síncrono) · order pide a catalog descontar stock · si algo falla DESPUÉS → **compensación**: order le pide a catalog **restaurar** el stock.

Decís:

«La respuesta tiene dos piezas. La primera es Feign: nos deja llamar a otro servicio como si fuera un método nuestro, de forma síncrona. Entonces order le pide a catalog que descuente el stock. La segunda pieza es la saga con compensación. La idea es: descuento el stock primero; si después algo falla al crear la orden, no puedo hacer un rollback automático, así que lo deshago a mano: order le pide a catalog que RESTAURE el stock. Eso es la compensación, el rollback distribuido hecho a mano. Y esto lo probamos con tests: el caso feliz, el caso sin stock, y el caso de compensación.»

Slide 16 — Cupón de descuento (integración real)

En pantalla: `couponCode` en el checkout · order pide el descuento a **promo por Feign** · `total = subtotal - descuento + envío` · el descuento es REAL (lo decide el backend).

Decís:

«Y sumamos los cupones, que es otra llamada Feign en el mismo checkout. Cuando el usuario pone un código de cupón, order le pregunta al servicio de promociones cuánto es el descuento. El total se calcula como subtotal menos descuento más envío. Y lo importante: el descuento es REAL, lo decide el backend. El frontend muestra una vista previa, pero la plata final la define el servidor, no el navegador.»

👉 **Traspaso:** «Y ahora Sergio les cuenta la otra forma en que se comunican los servicios, con mensajería, más el pago y las promos.»

🔴 Pausa — ahora hablan otros

Acá hablan **Sergio** (17–19) y **Jason** (20–21). **Tu próximo turno es el Slide 22, la demo.** Jason cierra el slide 21 diciendo "*le paso a Cristofer que va a hacer la demo en vivo*" — esa es tu entrada. **Tené el stack ya levantado antes de empezar la charla.**

Slide 22 — Demo en vivo

En pantalla: (sin texto, o los pasos) Login → catálogo → reseña → carrito → checkout con cupón → pagar → notificación en el log.

Decís:

«Bueno, basta de slides, vamos a verlo funcionando. Les voy a mostrar el flujo completo: me logueo, entro al catálogo, dejo una reseña en un producto, lo agrego al carrito, voy al checkout y aplico un cupón, pago la orden, y vamos a ver cómo aparece la notificación en el log del servicio de notificaciones.»

⚠️ **Antes de la charla:** *stack levantado (modo jars o Docker), usuario y producto de prueba listos, y el log de notification-service visible.* **Plan B: video grabado.**

Slide 23 — Conclusiones y aprendizajes

En pantalla: Patrones logrados: discovery, gateway, JWT stateless, Feign + saga, RabbitMQ, Docker · Trade-offs · Gotchas resueltos.

Decís:

«Y para cerrar la parte técnica, ¿qué aprendimos? Logramos los patrones que nos propusimos: descubrimiento con Eureka, el gateway, JWT stateless, Feign con saga, RabbitMQ y Docker. Pero también aprendimos que lo distribuido trae problemas nuevos: hay trade-offs, como que el token stateless es rapidísimo pero no podés dar de baja a alguien al instante. Y nos topamos con varios gotchas que tuvimos que resolver. Lo distribuido no es gratis, pero entendimos cómo manejarlo.»

👉 **Traspaso:** «Y para el cierre final, le dejo la palabra a Sergio.»

✅ **Listo.** *La demo es tu momento estrella — ensayala hasta que te salga sola.*